

Surgical Instrument Detection and Tracking using YOLO11 and ByteTrack

Project Report

Dataset: hSDB-instrument

Model: YOLO11s with Transfer Learning

Tracking: ByteTrack

Framework: Ultralytics YOLO / PyTorch

Sima Hosseini Parsa

2026

1. Introduction

Surgical video analysis has become an important application of computer vision and deep learning, particularly for the development of intelligent systems that can automatically interpret surgical procedures. The detection and localization of surgical instruments in endoscopic videos are fundamental tasks in this area, as reliable identification of instruments can provide the basis for higher-level video analysis and automated surgical scene understanding.

Surgical endoscopic videos present challenging visual conditions for object detection. Multiple instruments may appear simultaneously within the same frame, their appearance can change during motion, and instruments may partially overlap or become difficult to distinguish from surrounding structures. These characteristics make accurate and consistent instrument localization an important computer vision problem.

In this project, a deep learning-based object detection pipeline is developed using YOLO11 for surgical instrument detection. The hSDB-instrument dataset is prepared from COCO-format annotations and converted into the YOLO format. Different annotated components belonging to the same surgical instrument are merged into a single instrument-level class to simplify the detection task and provide a suitable representation for subsequent tracking. The resulting dataset contains 20 instrument classes.

A pretrained YOLO11s model is then fine-tuned on the prepared surgical instrument dataset. Model performance is evaluated using Precision, Recall, mAP@50, and mAP@

50–95, together with confusion matrices.

Finally, the trained detector is integrated with ByteTrack to extend frame-level detection to multi-object tracking. The detected surgical instruments are assigned tracking identities and followed across consecutive video frames, allowing the system to produce a visual tracking output from an endoscopic video.

2. Dataset

The project uses the hSDB-instrument dataset, which contains surgical video data from laparoscopic cholecystectomy and robotic gastrectomy procedures [1]. The dataset provides bounding-box annotations for surgical instruments and includes real, synthetic, and domain-randomized data.

For this project, the real-data subsets from Chole and Gastric procedures are used. The original annotations are provided in the COCO format and are processed to prepare the data for the object detection task.

The original annotations represent different components of surgical instruments, including heads, bodies, wrists, clips, and tips. Since the objective of this project is instrument-level detection and tracking, these components are grouped into their corresponding instrument categories.

After dataset preparation and class merging, the final dataset used in this project contains 20 instrument classes. Ligasure is excluded because it appears in the validation annotations but has no corresponding training instances and therefore cannot be used as an independently trained class.

3. Methodology

The overall workflow of the proposed surgical instrument detection and tracking pipeline is illustrated in Figure 1. The pipeline consists of dataset preparation, annotation conversion and validation, YOLO11 model training and evaluation, followed by ByteTrack-based object tracking and visualization.

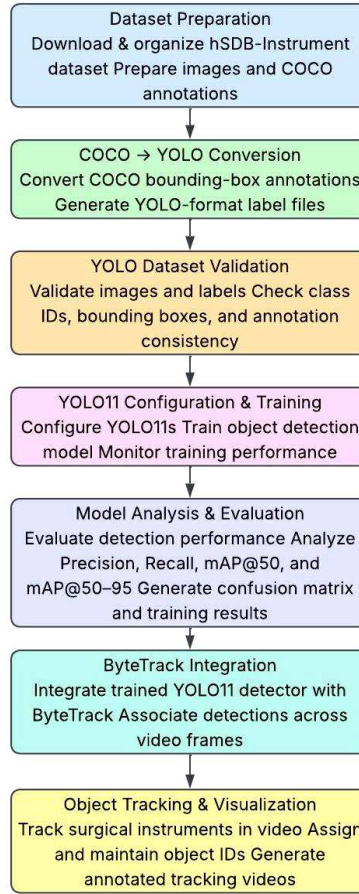


Figure 1. Overall pipeline of the surgical instrument detection and tracking system.

3.1. Dataset Preparation

The dataset preparation stage converts the original COCO-formatted annotations into a YOLO-compatible dataset.

First, the COCO annotation files corresponding to the Chole and Gastric training and validation subsets are loaded. The category information is extracted from the annotation files and used to identify the surgical instrument classes.

Because the original annotations represent different components of instruments separately, a class-merging procedure is applied. Component suffixes such as `_head`, `_body`, `_wrist`, `_clip`, and `_tip` are removed so that different components belonging to the same instrument are represented by one unified class.

The resulting annotations are then converted from COCO bounding boxes into the YOLO annotation format. For each object, the bounding box coordinates are converted into normalized center coordinates and normalized width and height values. The resulting

labels follow the YOLO format:

class_id, x_center, y_center, width, height

The conversion process is applied to all training and validation images from both Chole and Gastric subsets. Image names are prefixed with chole_ or gastric_ to avoid naming conflicts when combining the datasets.

The final dataset is organized into separate image and label directories for training and validation. Image-label correspondence is also checked to identify missing labels or orphan label files.

Finally, the generated YOLO annotations are validated by checking annotation format, class ID ranges, and normalized bounding box coordinates. The validation procedure checks every annotation and reports invalid entries.

3.2. Image Preprocessing

The notebook does not implement a separate manual image preprocessing pipeline such as image normalization, color transformation, or filtering. Instead, image preprocessing is handled by the Ultralytics YOLO training and inference pipeline.

The training, validation, and prediction stages are configured with an input image size of 640×640 pixels. The YOLO framework applies the required image resizing and preprocessing operations internally during these stages.

No additional manual preprocessing operations are applied to the input images before they are provided to the YOLO pipeline.

3.3. Data Augmentation

No independent or manually defined data augmentation pipeline is implemented in the notebook. The training procedure relies on the augmentation and preprocessing mechanisms provided internally by the Ultralytics YOLO training framework rather than defining custom augmentation operations in the project code.

Therefore, augmentation parameters are not reported as individually configured experimental variables in this project.

3.4. Model Architecture

The object detection model used in this project is YOLO11s, loaded through the Ultralytics YOLO framework. YOLO11 is a one-stage object detection architecture designed to perform object localization and classification efficiently within a unified detection pipeline.

The yolo11s.pt pretrained model is loaded as the initial model, after which it is trained on the custom surgical instrument dataset.

The final detection configuration contains 20 instrument classes, which are defined in the generated data.yaml configuration file together with the training and validation image paths.

3.5. Transfer Learning

Transfer learning is used by initializing the detector with pretrained YOLO11s weights instead of starting the training with randomly initialized weights. The notebook loads the pretrained yolo11s.pt weights and uses them to initialize the model.

The pretrained model is subsequently fine-tuned using the prepared surgical instrument dataset. This approach provides a practical initialization for adapting the detector to the target surgical domain.

3.6. Training Strategy

The YOLO11s model is trained using the prepared 20-class dataset. Training is configured for a maximum of 100 epochs with an image size of 640×640 pixels and a batch size of 16.

Early stopping is enabled with a patience value of 20 epochs. This allows the training process to terminate when validation performance does not improve for the specified number of epochs. Four workers are used for data loading, and GPU device 0 is selected for training.

The model is initialized using pretrained weights, and the best checkpoint generated by the Ultralytics training process is saved during training. The resulting best.pt weights are subsequently used for model evaluation and tracking.

3.7. Model Evaluation

The trained model is evaluated using the best saved checkpoint on the validation dataset. The evaluation is performed using an image size of 640×640 and a batch size of 16.

The primary detection metrics are Precision, Recall, mAP@50, and mAP@50–95.

The training process also records box loss, classification loss, and distribution focal loss for both the training and validation sets. These values are included in the training results plot to monitor the learning behavior of the model throughout the training process.

Additional evaluation outputs include a standard confusion matrix and a normalized confusion matrix.

4. Experimental Setup

4.1. Development Environment

The project is implemented in Python using the Ultralytics YOLO framework together with PyTorch and supporting computer vision and data processing libraries. The notebook imports Ultralytics YOLO, PyTorch, OpenCV, pandas, Matplotlib, and other utilities required for dataset preparation, training, evaluation, and tracking.

The notebook uses Python 3.12 as its kernel language version. PyTorch 2.11.0 with CUDA 12.8 support is used for GPU-accelerated computation.

GPU acceleration is checked through PyTorch before training, and GPU device 0 is selected for the YOLO training and validation processes.

4.2. Training Configuration

The main training configuration is summarized below:

Parameter	Configuration
Model	YOLO11s
Initialization	Pretrained weights
Number of classes	20
Input image size	640 × 640
Maximum epochs	100
Batch size	16
Early stopping patience	20
Data loader workers	4
Device	GPU 0

These parameters are directly specified in the YOLO training configuration in the notebook.

4.3. Training Process

The training process begins with the prepared YOLO dataset and its corresponding data.yaml configuration file. The pretrained YOLO11s model is then fine-tuned on the 20-class surgical instrument dataset.

During training, the framework records training and validation losses and detection metrics. The generated training results are subsequently inspected through the YOLO

training results file and plots. The best model checkpoint is identified from the generated weights directory and is used for the subsequent evaluation stage.

5. Experiments

The training experiment resulted in a trained YOLO11s model, which was subsequently evaluated on the validation set and used as the detection component of the tracking pipeline.

5.1. YOLO11s Training

The primary experiment consists of training the pretrained YOLO11s model on the prepared hSDB-instrument dataset. The model is trained using the configuration described in Section 4, with a maximum of 100 epochs, 640×640 input resolution, batch size 16, and early stopping patience of 20 epochs.

The purpose of this experiment is to adapt the pretrained detector to the surgical instrument domain and establish a detection model capable of recognizing the 20 target instrument classes.

5.2. Detection Confidence Threshold

The trained model is used to generate predictions on the validation images. A confidence threshold of 0.25 is used when generating prediction visualizations on the validation images. The resulting predictions are saved for visual inspection. This threshold is used only for the prediction stage and is not applied to the calculation of the primary validation metrics.

The confidence threshold determines the minimum confidence required for a predicted detection to be retained. This parameter is particularly relevant in surgical scenes containing multiple instruments, where excessively low thresholds may increase false detections while excessively high thresholds may remove valid detections.

5.3. ByteTrack Integration and Tracking

After training and validation, the best YOLO11s model is integrated with ByteTrack for multi-object tracking. The detector provides object detections for individual video frames, while ByteTrack associates detections across consecutive frames and assigns persistent tracking IDs.

Three confidence thresholds of 0.15, 0.25, and 0.35 were tested for the tracking stage. The resulting tracking outputs were qualitatively compared, and a confidence threshold of 0.25 was qualitatively judged to provide the most suitable tracking output among the tested thresholds. Therefore, a confidence threshold of 0.25 was used for the final tracking output. The tracking pipeline uses an input image size of 640×640 pixels with persistent tracking enabled.

The resulting tracked video is saved for qualitative visualization.

5.4. Final Model Selection

The final detection model is selected from the best checkpoint generated during YOLO11s training. The saved best.pt weights are loaded and used for validation, prediction, and the subsequent ByteTrack tracking stage.

This ensures that the same selected model is used as the detection component of the final detection-and-tracking pipeline.

6. Results

6.1. Performance Metrics

The trained YOLO11s model is evaluated on the validation dataset using Precision, Recall, mAP@50, and mAP@50–95 as the primary object detection metrics.

The evaluation results provide an overall assessment of the detector's ability to identify and localize surgical instruments in the validation dataset. The training results shown in Figure 2 visualize the progression of the detection metrics and loss components throughout the training process.

Metric	Value
Precision	0.7473
Recall	0.5801
mAP@50	0.6172
mAP@50–95	0.4408

Table 1. Final validation performance of the selected best.pt checkpoint (best epoch: 24).

6.2. Training Curves

The YOLO training process generates a results plot containing training and validation losses together with object detection performance metrics. The plot includes box loss, classification loss, distribution focal loss, Precision, Recall, mAP@50, and mAP@50–95 across the training epochs.

The training and validation loss curves provide an overview of the model's learning behavior during training, while Precision, Recall, mAP@50, and mAP@50–95 show the changes in detection performance across the training epochs.

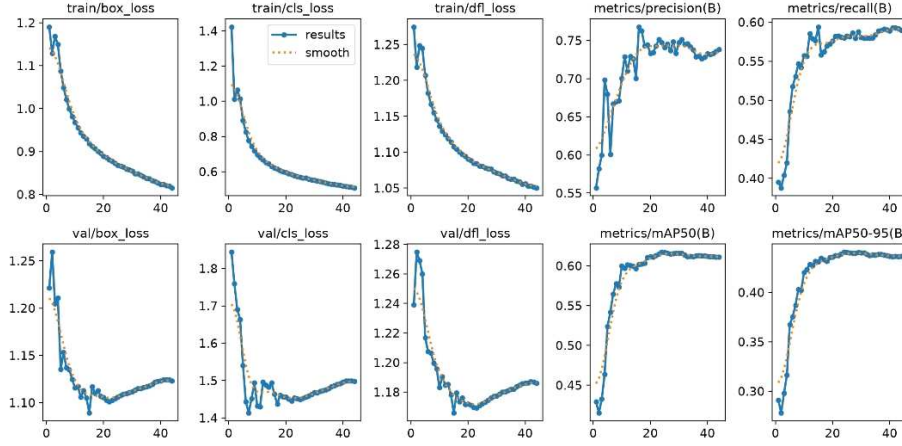


Figure 2. Training and validation losses and object detection performance metrics during YOLO11s training.

The figure provides a visual overview of the model's training behavior, including the evolution of detection metrics and loss components throughout the training process.

6.3. Confusion Matrix

A confusion matrix is generated during validation to provide a class-level view of the detection and classification behavior of the object detector across the 20 instrument classes, including background-related predictions. A normalized version of the confusion matrix is also generated to provide a relative view of class-level prediction behavior.

The confusion matrix provides additional information beyond the overall detection metrics by showing which instrument classes are more frequently confused with one another.

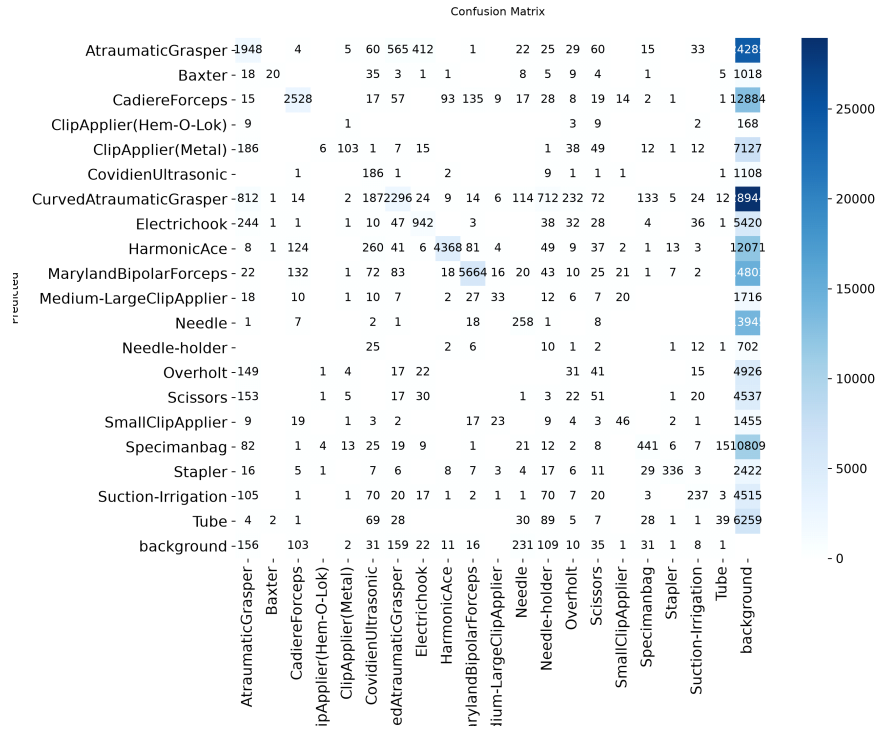


Figure 3. Confusion matrix of the YOLO11s model on the validation dataset.

The confusion matrix provides a class-level view of the model's detection behavior and highlights the distribution of correct and incorrect predictions among the surgical instrument classes.

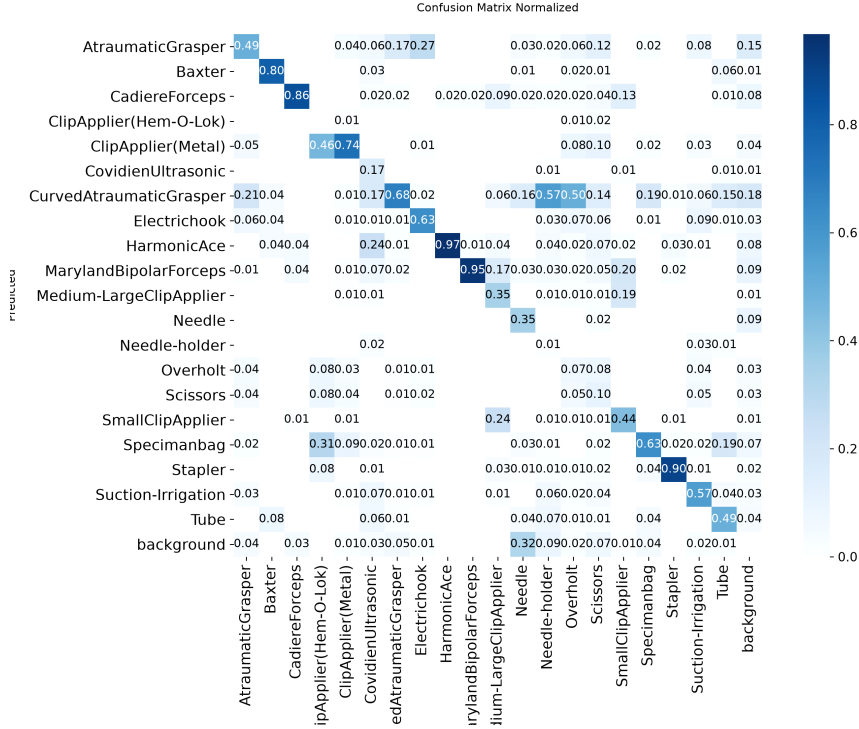


Figure 4. Normalized confusion matrix of the YOLO11s model on the validation dataset.

The normalized confusion matrix provides a relative representation of the prediction distribution across the instrument classes, facilitating comparison between classes with different numbers of samples.

6.4. Tracking Results

The trained YOLO11s detector is combined with ByteTrack to track surgical instruments across video frames. The resulting video contains bounding boxes and tracking identities for detected instruments.

A shortened segment of an externally sourced surgical video and its corresponding tracking output are provided for qualitative visualization. The input video was obtained from Video S1 of [Kawai et al., 2025] and was used solely as an external video source for demonstrating the tracking pipeline. The external video is not part of the hSDB-Instrument dataset and was not used for model training or validation. The video source is available in the supplementary materials of the corresponding publication [2].

[\[TRACKING VIDEO REFERENCE\]](#)

7. Discussion

The developed pipeline demonstrates an end-to-end approach for surgical instrument

detection and tracking using a combination of YOLO11 and ByteTrack. The workflow begins with the preparation of COCO annotations and continues through class merging, annotation conversion, dataset validation, model training, object detection evaluation, and multi-object tracking.

An important aspect of the project is the transformation of component-level annotations into instrument-level classes. The original dataset contains multiple components belonging to individual instruments, while the final objective is to detect and track complete instruments. Merging these components simplifies the target representation and makes the resulting detections more suitable for integration with a tracking algorithm.

The use of pretrained YOLO11s weights provides a transfer-learning-based starting point for the surgical instrument detection task. The model is then adapted to the custom 20-class dataset through supervised training.

The evaluation pipeline combines overall detection metrics with confusion matrices, allowing the detection behavior across the instrument classes to be examined.

The integration of ByteTrack extends the system from independent frame-level detections to temporal tracking. This is particularly useful for surgical video analysis because the same instrument can be followed across consecutive frames rather than being treated as a new detection in every frame.

However, the current results should be interpreted primarily as a portfolio-level implementation demonstrating the complete detection and tracking workflow. A more extensive experimental study would require additional controlled comparisons and dedicated tracking evaluation metrics.

8. Limitations

Several limitations should be considered when interpreting the results of the project.

First, the final dataset contains 20 classes after merging instrument components and excluding Ligasure because it has no corresponding training instances. Consequently, the model does not evaluate Ligasure as an independent target class.

Second, the current notebook does not implement a separate custom image preprocessing or augmentation pipeline. The training process relies on the preprocessing and augmentation mechanisms provided by the Ultralytics YOLO framework rather than on manually defined augmentation operations.

Third, the model evaluation focuses primarily on object detection metrics. Although the detection model is integrated with ByteTrack to generate multi-object tracking results, dedicated multi-object tracking evaluation metrics are not implemented in the current pipeline.

Finally, the current implementation is based on a single YOLO11s training configuration.

The notebook does not contain a systematic comparison between different YOLO11 model sizes or a controlled hyperparameter study.

9. Future Work

Several extensions could improve the current system.

A first direction would be to investigate larger YOLO11 model variants and compare their detection accuracy and computational cost with the YOLO11s baseline.

A second direction would be to introduce systematic experiments involving data augmentation and hyperparameter optimization. Different augmentation strategies, learning rates, batch sizes, and training schedules could be evaluated to determine their effect on detection performance.

Another potential improvement would be to evaluate the tracking component using dedicated multi-object tracking metrics. This would provide a more comprehensive assessment of identity preservation and tracking consistency beyond the current qualitative evaluation.

The system could also be extended toward real-time deployment. Optimization of inference speed, model size, and hardware utilization could make the pipeline more suitable for real-time surgical video analysis.

Finally, additional surgical datasets and more diverse video sequences could be incorporated to evaluate the robustness and generalization of the detection and tracking system across different surgical procedures and visual conditions.

10. Conclusion

This project developed an end-to-end pipeline for surgical instrument detection and tracking using YOLO11 and ByteTrack. The hSDB-instrument dataset was prepared by loading the original COCO annotations, merging instrument components into instrument-level classes, converting the annotations to YOLO format, constructing the YOLO dataset structure, and validating the generated annotations. The final dataset contains 20 surgical instrument classes.

A pretrained YOLO11s model was fine-tuned on the prepared dataset using a 640×640 input resolution, batch size of 16, and a maximum of 100 training epochs with early stopping. The trained model was evaluated using Precision, Recall, mAP@50, mAP@50–95, and confusion matrices.

The final detection model was subsequently integrated with ByteTrack to enable multi-object tracking in endoscopic video. Tracking identities were assigned to detected instruments and maintained across consecutive video frames. The resulting tracking output was saved as a video for qualitative visualization.

Overall, the project demonstrates a complete computer vision workflow that combines dataset preparation, deep learning-based object detection, quantitative evaluation, and multi-object tracking for surgical video analysis. The resulting pipeline provides a foundation for further development toward more robust and real-time surgical instrument analysis systems.

11. References

- [1] Yoon, J., Lee, J., Heo, S., et al. (2021). hSDB-instrument: Instrument Localization Database for Laparoscopic and Robotic Surgeries. *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, 393–402.
- [2] Kawai, H., et al. (2025). Planarization, Squarization, and Verticalization for Optimal Surgical View Exposure in Laparoscopic Cholecystectomy. *Asian Journal of Endoscopic Surgery*.